Value Attribution in Agentic Systems: Shapley, Ledgers, and the Contribution of Code

Zachary F. Mainen

Champalimaud Research

March 2026 · HAAK Technical Report

ABSTRACT

The problem of fairly distributing value among contributors to a shared outcome is a classic problem in cooperative game theory, solved in principle by the Shapley value. Recent work in machine learning — Data Shapley (Ghorbani & Zou, 2019) and its successors — has made this tractable for data contributions to ML models. But in agentic systems like HAAK, value is created not only by data but by code, policy decisions, and method documents: contributions that standard Data Shapley cannot handle because it treats everything except data as neutral infrastructure. We describe an extension of the Shapley framework to heterogeneous contribution types, enabled by a provenance ledger (Filix) that records the marginal effect of every contribution on system performance. We discuss the relationship to Ocean Protocol's data tokenisation approach, the specific challenges of temporal attribution — when did a contribution matter, and for how long? — and the open problems that remain. The framework has an immediate application in art intelligence platforms where artists, galleries, and data providers all contribute to a shared recommendation model, and where the incentive to share depends on whether sharing is demonstrably compensated.

1. Introduction

When a sale occurs in a platform that uses a machine-learning recommendation engine, a natural question arises: who contributed to it? The artist contributed the work that was sold. The gallery contributed the market context and collector relationships. A data provider contributed historical auction records that trained the pricing model. An engineer contributed the algorithm that improved recommendation quality by 15%. A policy negotiator contributed the data-sharing agreement that unlocked 3 million behavioral profiles. A researcher contributed the method document that defined how taste clusters are extracted from interaction data. The sale was a joint outcome of all of these contributions. The question of how to distribute credit — and compensation — among them is not rhetorical. It determines whether any of them will contribute in the future.

Standard industry practice offers three approaches, none of them satisfying. Revenue sharing by contract is opaque: fractions are negotiated by leverage, not by contribution, and the party with the most leverage captures most of the value. The platform-keeps-everything model is the most common outcome of that negotiation, which simply reproduces the incentive problem: if you can't be compensated for what you contribute, you don't contribute. Voluntary royalties — the artist-friendly alternative sometimes proposed by advocacy organisations — are not robust to competitive pressure and don't scale to the data and code contributions that actually drive model performance.

The failure of all three approaches has a common root: they don't track actual contribution. They rely on pre-specified rules or post-hoc negotiation rather than on a measurement of what each party actually added to the system's value. This creates perverse incentives: a gallery that withholds its market data from a shared training corpus is protecting itself from exploitation. It is also, inadvertently, degrading the model that produces recommendations it depends on. The incentive to withhold and the incentive to contribute point in opposite directions, and the absence of a trustworthy attribution mechanism is what keeps them misaligned.

Cooperative game theory offers a principled alternative. The Shapley value (Shapley, 1953) is the unique solution to the problem of fairly distributing the value of a coalition game among its members. It has four properties — efficiency, symmetry, the null-player axiom, and linearity — that together characterise "fair" attribution. Ghorbani and Zou (2019) showed that it can be applied directly to the problem of data attribution in machine learning, by treating model performance as the value function and data subsets as the coalitions. The resulting Data Shapley score assigns each training example the average marginal contribution it makes to model performance across all possible subsets of the training set.

The thesis of this paper is that the Shapley framework extends naturally from data to the full range of contributions in an agentic system — code, policy decisions, and method documents alongside data — if and only if the system maintains a provenance ledger that records the marginal effect of every contribution on system performance. Without such a ledger, the computation is not tractable. With it, the attribution problem reduces to a query over a structured history of the system's development. We describe the Filix ledger in HAAK as an implementation of this infrastructure, discuss its relationship to Ocean Protocol's data economy approach, and identify the open problems that a complete solution must address.

2. The Shapley Value

2.1 Definition and axioms

Let $N = \{1, \ldots, n\}$ be a set of players and $v : 2^N \to \mathbb{R}$ a characteristic function assigning to each coalition $S \subseteq N$ a real-valued payoff $v(S)$, with $v(\varnothing) = 0$. The Shapley value of player i is:

$$\phi_i(v) = \Sigma_{S \subseteq N \setminus \{i\}} \left[|S|!(|N|-|S|-1)! \,/\, |N|!\right] \cdot \left[v(S \cup \{i\}) - v(S)\right]$$

The term $v(S \cup \{i\}) - v(S)$ is the marginal contribution of player i to coalition S: the difference in payoff with and without i, given that the other members of S are already present. The Shapley value averages this marginal contribution over all possible orderings in which the players might join the coalition — equivalently, over all subsets S of the players who joined before i. The coefficient $[|S|!(|N|-|S|-1)!/|N|!]$ is the probability that exactly this coalition preceded i's arrival under a uniform distribution over orderings.

Four axioms characterise this value uniquely. *Efficiency* requires that the sum of all players' Shapley values equals the total value of the grand coalition: $\Sigma_i \phi_i(v) = v(N)$. This ensures that the distribution is exhaustive — nothing is withheld. *Symmetry* requires that if two players make identical marginal contributions in all coalitions, their Shapley values are equal. This is the formal statement of "equal pay for equal contribution." The *null player* axiom requires that if a player makes zero marginal contribution to every coalition, their Shapley value is zero. This eliminates free riders. *Linearity* (or additivity) requires that the Shapley value of a sum of games is the sum of the Shapley values: if the overall value can be decomposed into independent components, each player's attribution decomposes accordingly.

These four axioms together make the Shapley value not just one possible attribution rule but the uniquely justified one: Shapley (1953) showed that it is the only function satisfying all four simultaneously. This is what distinguishes it from ad hoc revenue-sharing agreements, which typically satisfy none of them.

2.2 Computation

The direct computation of the Shapley value requires evaluating $v(S)$ for all 2^n subsets S, which is exponential in n. For large datasets — millions of training examples, thousands of code commits, hundreds of policy decisions — exact computation is intractable. Two families of approximation make it practical.

The first is Monte Carlo sampling. Because the Shapley value is an expectation over orderings, it can be approximated by sampling random permutations and computing the marginal contribution of each player in each sampled ordering. The approximation converges at rate $O(1/\sqrt{T})$ where T is the number of sampled permutations, with

computable confidence intervals. For most attribution applications, T = 10,000 to 100,000 permutations gives sufficient precision, and permutation evaluations are embarrassingly parallelisable.

The second family exploits structure in the value function. Data Shapley (Ghorbani & Zou, 2019) applies the Monte Carlo approach to ML data attribution, replacing v(S) with the performance of a model trained on subset S. Since retraining a model for every subset is prohibitively expensive, the authors use K-nearest-neighbour approximations and performance-sensitive subsampling strategies that reduce the number of retraining runs by orders of magnitude. Beta-Shapley (Kwon & Zou, 2022) extends this by introducing a family of weighted Shapley values parameterised by a Beta distribution, which concentrate attention on coalitions of particular sizes — a design choice that improves sample efficiency when small coalitions are more informative than large ones, as is often the case in data attribution.

Influence functions (Koh & Liang, 2017) offer a third approach: instead of retraining on subsets, they use the model's gradient at a training example to estimate the change in loss if that example were removed. This is computable at query time without any retraining, at the cost of an approximation error that scales with the non-convexity of the loss landscape. For production systems where attribution must be computed quickly and continuously, influence functions provide a useful first-order approximation that can be refined by occasional full Shapley computations.

SHAP (Lundberg & Lee, 2017) — which has become the dominant explainability framework in industry — operates at a different level of analysis than the others. It computes feature-level attribution within a single prediction: how much did each input feature contribute to this output? This is distinct from data-level attribution across the training set: how much did each training example contribute to the model's overall performance? Both are Shapley-based, but they answer different questions. The former is useful for explaining individual decisions; the latter is necessary for compensating data contributors. Both are relevant to the system we describe, but they should not be conflated.

2.3 The missing dimension: non-data contributions

Data Shapley handles training data. It treats the learning algorithm, the model architecture, the feature engineering pipeline, the governance rules that determined which data was collected, and the method documents that shaped what was extracted from that data as fixed infrastructure. These are assumed to be common knowledge, jointly owned by all parties, and therefore not attributable to any individual contributor. This assumption is reasonable when the system is a standard ML pipeline with a fixed algorithm applied to a dataset. It is false in an agentic system.

In HAAK, the algorithm is itself a product of contributions: software engineers, research scientists, and collaborating AI agents have each contributed code whose marginal effect on model performance is measurable and non-zero. The governance rules that determined

which data was collected are themselves the product of policy decisions made by identifiable agents. The method documents that define how features are extracted and how recommendations are ranked have been authored by specific contributors whose choices have downstream effects on every model trained under them. Treating all of this as infrastructure — as Data Shapley implicitly does — is not just analytically incomplete. It reproduces the incentive problem: if you know your algorithmic contribution will be absorbed into the infrastructure and not attributed, you have less reason to contribute it.

3. Ocean Protocol and the Data Economy

Ocean Protocol (Trentmann et al., 2022) approaches the attribution problem from a different angle: rather than computing Shapley values and distributing revenue, it tokenises data itself. Each dataset is represented by a datatoken — an ERC-20 token on Ethereum whose ownership corresponds to the right to access the data. Revenue from data usage flows automatically to token holders via smart contracts, without requiring any centralised computation of marginal contributions. Compute-to-data (C2D) architecture allows algorithms to run against encrypted data on the data provider's infrastructure, so that the data never leaves the provider's control while still generating value for external parties.

What Ocean gets right is substantial. Data as a first-class asset — with ownership, pricing, and revenue distribution handled on-chain — solves the trust problem that makes voluntary royalties unworkable. If attribution is encoded in a smart contract rather than a bilateral agreement, neither party needs to trust the other's good faith. The C2D architecture also addresses the privacy problem: data providers do not need to expose their data to benefit from its use. These are genuine advances over the negotiated-contract model.

What Ocean treats as infrastructure, however, is everything that is not data. The algorithm that trains on the data, the pipeline that preprocesses it, the governance decisions that determine what data is eligible for inclusion, and the methods that define what features are extracted — all of this is implicitly assumed to be neutral. Ocean's tokenisation model has no mechanism for attributing value to a code contribution, because in Ocean's ontology, code is infrastructure rather than a contributing asset.

This assumption reflects a particular view of where value originates. In many data markets — real estate transaction records, weather station measurements, genomics databases — the data really does carry most of the value, and the algorithms are indeed commodity tools. In an agentic art intelligence system, this is plainly false. The recommendation algorithm's architecture, its training objective, and its evaluation methodology together determine whether the system is useful at all. An undifferentiated Ocean token for a dataset of 100,000 artworks attributes zero value to the researcher who spent a year developing the aesthetic embedding that makes those artworks computationally comparable — even if that embedding is, demonstrably, the difference between a system artists want to use and one they don't.

The deeper issue is that in a collaborative agentic system, algorithms emerge from interactions between human contributors, AI agents, and accumulated method documents. They do not have a single author or a fixed codebase. Their authorship is distributed and dynamic. Ocean's model — in which each asset has an owner who holds the corresponding token — struggles with contributions that are inherently collective and path-dependent. The Shapley framework, as we will argue, handles this case naturally: it does not require identifying an author, only measuring a marginal effect.

4. Heterogeneous Contribution Types

An agentic system like HAAK creates value through at least four distinct types of contribution: materials (data), code, policy decisions, and method documents. Each type requires a distinct analysis, but all can be integrated into a common Shapley framework once the provenance ledger is in place.

4.1 Materials (data)

The materials case is where Data Shapley applies directly. Every data point contributed to the system — an artwork deposit, an auction result, a user interaction event, a market price record — has a Shapley value equal to its average marginal contribution to model performance across all subsets of the training corpus it might belong to. In an art intelligence platform, this includes the artworks themselves, the metadata associated with them, and the behavioral and transactional records generated by user interactions.

The practical challenge is scale: a system with millions of training examples cannot evaluate exact Shapley values. Beta-Shapley (Kwon & Zou, 2022) provides a principled approximation that is particularly well-suited to this context, because it allows the analyst to weight subsets of different sizes differently. Empirically, in data attribution problems, small coalitions tend to be more informative than large ones — adding one example to a 10-example training set reveals more about its marginal contribution than adding one example to a million-example set, because at large scale the contribution is typically diluted to near-zero. Beta-Shapley's parameterisation makes this size-dependence explicit and adjustable.

Attribution of data contributions should also respect temporal structure. Data contributed early, when the training corpus was small, typically has a larger marginal effect than data contributed later, when the corpus was already large and the model had already generalised. Standard Shapley is atemporal: it treats all orderings as equally probable. We return to this in Section 5.

4.2 Code

A code contribution's Shapley value is, formally, identical to a data contribution's Shapley value: it is the average marginal contribution of the code change to model performance across all subsets of contributions it might be included in. The value function $v(S)$ is now

the performance of the model built using the subset S of code commits, rather than trained on a subset S of data points. The computation is structurally the same; the interpretation differs.

What makes code attribution practically different is the size of the space. A large codebase might have thousands of commits, but rarely millions. And each commit typically has a non-negligible effect on model performance — unlike data points, where individual contributions are often tiny, individual code changes can move the performance metric by several percentage points. This means that exact Shapley is often tractable for code when it is not for data: the coalition space is small enough to enumerate meaningfully, and the signal-to-noise ratio of each evaluation is high enough to be informative.

The natural value function for code attribution is a performance metric — accuracy, recommendation quality, revenue per session — evaluated on a held-out test set. The model is "rebuilt" from each subset of commits by replaying the commit history on the subset: this is well-defined because version control gives commits a partial order. The Shapley value of a particular commit is then its average marginal effect on this performance metric, averaged over all orderings consistent with the commit's position in the dependency graph. Topological sorting of the dependency graph reduces the permutation space and makes the computation more tractable.

4.3 Policy

A policy contribution is a governance decision that changes what data is available to the system. A data-sharing agreement with a major auction house unlocks transaction records. A privacy policy amendment expands what user behavioral data is retained. A constitutional governance decision establishes the framework under which all future data contributions are made. Each of these is a policy contribution with a measurable downstream effect: the data it enables.

Policy attribution has a natural two-stage structure. First, identify the data that was enabled by the policy: the set of training examples that are present because of this decision and absent in its counterfactual. Second, apply standard Data Shapley to this enabled dataset to compute its contribution to model performance. The policy's Shapley value is then (approximately) the aggregate Shapley value of its enabled data — it deserves credit for enabling the contribution that the data makes.

This two-stage structure is a simplification. A more precise treatment would note that policies do not just enable data; they shape what data is collected, how it is labeled, and how it is structured. A policy that mandated standardised auction metadata formats might have increased the usefulness of existing data without increasing its quantity. Attributing this effect requires a more granular model of how policy choices propagate through data collection and preprocessing pipelines. The ledger, as described in Section 5, provides the necessary instrumentation.

4.4 Method

Method documents are the hardest case. A method document — in HAAK's ontology, a formal description of how a process should be conducted — shapes how the model is trained, how features are extracted, how recommendations are ranked, and how model performance is evaluated. Its influence is diffuse and path-dependent: a method document that established a three-cluster model of aesthetic taste may have shaped every model trained for the next five years, but its influence on any individual training run is difficult to isolate.

The difficulty is causal, not merely computational. When a method document is in effect, everything downstream of it is potentially influenced. But many of the downstream effects would have occurred anyway under different methods, and some of the method's effects compound with code and data contributions in ways that make clean counterfactuals hard to specify. What would the system have done without this method? The answer depends on what would have replaced it — a different method, no method at all, or a legacy practice that was itself shaped by earlier methods.

Despite these difficulties, method attribution is not impossible. The ledger records which method was active at each training step. By comparing the performance of models trained under this method with models trained under the preceding or alternative method — a comparison that the ledger's versioned history makes possible — the marginal effect of the method transition can be estimated. The estimate is noisy, because method transitions often co-occur with other changes, but the longitudinal structure of the ledger allows confounders to be identified and, in some cases, controlled for. We treat method attribution as a partially solved problem: the framework is in place, the causal identification strategy requires case-by-case analysis, and the remaining uncertainty should be acknowledged rather than papered over.

5. The Filix Ledger as Attribution Infrastructure

5.1 What Shapley requires

Computing Shapley values for any type of contribution requires the ability to evaluate $v(S \cup \{i\}) - v(S)$ for all relevant subsets S. In the ML attribution setting, this means evaluating the model's performance before and after the inclusion of contribution i, across a representative sample of training configurations. This is not a lightweight requirement: it demands that the system maintain a complete history of its development, annotated with performance measurements, and that this history be queryable in a form that supports counterfactual computation.

Specifically, for each contribution c, the attribution infrastructure must be able to answer: what was the model's performance on a relevant evaluation metric at each point in time? What contributions were included at each point? What would performance have been if c

had been excluded, holding everything else constant? These are structured queries over a history of system states, and they require a data structure designed to support them.

5.2 Filix: HAAK's constitutional ledger

Filix is HAAK's provenance ledger. It is built on a Merkle-CRDT structure: a content-addressed directed acyclic graph where each node is a cryptographically signed state change, and the hash of each node commits to the full history of the system that produced it. Every contribution to the system — a data deposit, a code commit, a policy decision, a method update — is a signed entry in the ledger, linked to the ledger state that preceded it and carrying a cryptographic commitment to its content.

Every model version trained in the system is associated with a ledger state: a hash that identifies the exact set of contributions that were present when the model was trained. Every performance measurement is recorded against the ledger state of the model it evaluated. This gives the system a complete, cryptographically verifiable map from contributions to performance measurements.

The Merkle structure provides two properties essential for attribution. First, it is tamper-evident: any modification to a historical ledger entry invalidates all subsequent entries, because their hashes commit to the prior chain. This means that contributors can trust that their historical contributions will not be retroactively erased or modified to reduce their Shapley scores. Second, it is content-addressed: any two systems that produced the same ledger state are guaranteed to be identical, which enables verification of claimed performance measurements by independent parties.

The CRDT component handles the distributed case: when multiple agents — human contributors, AI agents, external data providers — are writing to the ledger simultaneously, the CRDT merge semantics ensure that concurrent writes are resolved consistently without requiring a central coordinator. This is essential for a system like HAAK, where contributions arrive asynchronously from parties who cannot always coordinate in real time.

5.3 From ledger to Shapley

With the Filix ledger in place, computing a contribution's Shapley value becomes a structured query over the ledger's history. The Monte Carlo approximation samples random orderings of contributions, computes the marginal performance effect of the target contribution in each ordering, and averages. The ledger provides the sampling distribution: each historical training run corresponds to a sample from the set of coalitions that were present at that point in the development history.

Concretely, for a data contribution c deposited at ledger state h_t, the Shapley computation proceeds as follows. The ledger is traversed to find all training runs that (a) occurred after h_t (so that c was available) and (b) included c in their training corpus. For each such run, the corresponding excluded run — the closest training run at the same ledger state but with

c held out — is either found in the history (if such a run exists) or estimated using influence functions on the closest matching run. The marginal effect of c is the performance difference between the included and excluded runs. These marginal effects are averaged, weighted by the inverse probability of observing the corresponding coalition size, to approximate the Shapley value.

This is the Monte Carlo Shapley approximation, with the ledger providing the empirical sampling distribution rather than a synthetic one. The approximation quality depends on the density of training runs in the ledger history: a system that retrains frequently provides a richer basis for estimation than one that retrains rarely. In practice, HAAK retrains on each significant contribution, producing a training-run density that makes Shapley approximations reliable within approximately a hundred samples.

5.4 Temporal Shapley

Standard Shapley is atemporal: it averages marginal contributions over all orderings with equal weight, implicitly treating early contributors no differently from late ones. This is wrong in a system that grows over time. A Zenodo data deposit that was essential to the first version of the model — when the training corpus was tiny and every data point had enormous marginal effect — deserves credit for enabling the model's existence, even if the same data deposited today would have negligible marginal effect against a background of millions of records.

Temporal Shapley addresses this by weighting orderings according to the system's causal dependency structure. Early contributions — those that appear deep in the ledger's dependency graph — are up-weighted relative to their share in a uniform distribution, because they were more likely to be in small coalitions where marginal effects are large. This is equivalent to using the system's actual developmental history, rather than a synthetic uniform distribution, as the sampling distribution for the Monte Carlo approximation.

The Filix ledger's directed graph structure makes temporal Shapley tractable: the causal depth of each contribution is simply its topological distance from the genesis node, and this distance is computable in linear time. Weighting schemes that reflect this distance can be applied directly to the Monte Carlo estimator. The result is a temporally adjusted Shapley value that gives early contributors the additional credit that their causal position in the system's history justifies.

6. Distribution Mechanism

The Shapley values computed from the ledger define a principled attribution. Translating that attribution into actual revenue distribution requires a mechanism. We describe an architecture in which sale events trigger automatic distribution, governed by a smart contract that reads attribution from the ledger.

When a sale occurs on the platform, the sale event triggers a ledger query that retrieves the Shapley values for all contributions currently registered in the system. These values are pre-computed incrementally — updated at each training run as new contributions arrive — so that sale-time distribution is a lookup rather than a computation. The smart contract distributes a defined fraction of the sale revenue — the attribution pool — to contributors in proportion to their Shapley scores.

The size of the attribution pool is not a technical question but a governance question. It is the fraction of revenue that the platform commits to returning to contributors, in contrast to what is retained for operations, infrastructure, and platform profit. In HAAK, the attribution pool fraction is a constitutional parameter: it is set by the platform's governance layer, amendable through the constitutional process, and recorded in the Filix ledger. This means that changes to the pool fraction are themselves contributions to the governance record, attributable to the agents who proposed and ratified them.

Practical implementation requires that Shapley approximations be kept current. Because recomputing Shapley values from scratch at every training run is expensive, the system maintains a running Shapley estimate that is updated incrementally as new contributions arrive. Each new training run adds a new data point to the Monte Carlo estimator for every contribution it includes, updating the running average. The computational cost of each update is proportional to the number of contributions in the current training run, not to the total number of contributions in the system — which keeps the update cost manageable as the system grows.

7. Open Problems

Several important problems remain open. We state them honestly rather than suggesting that they are solved.

The **cold-start attribution problem** arises because early contributors cannot be fairly attributed when the model does not yet exist. A data provider who deposits artworks before the first model is trained has made a contribution whose Shapley value cannot be computed until training runs begin producing performance measurements. The partial solution is retroactive attribution: the ledger records the contribution's timestamp, and when performance measurements become available, Shapley values are computed retroactively and paid at defined milestones. This is technically straightforward but requires upfront agreement on the milestone schedule — an agreement that itself needs to be recorded in the ledger as a policy contribution.

Collusion poses a subtler problem. If a cartel of contributors can coordinate their data deposits to maximize each other's apparent Shapley values — for instance, by depositing carefully chosen complementary data points whose joint marginal contribution is large even though individually each is small — they can potentially inflate their collective attribution at the expense of independent contributors. The null player axiom provides some protection: a

player whose marginal contribution is genuinely zero cannot have a positive Shapley value regardless of how the coalition is structured. But a cartel whose contributions are genuinely valuable, and whose coordination merely ensures that their joint value is captured by their Shapley scores rather than diluted, is not obviously exploiting the system. The boundary between legitimate complementary contribution and collusive inflation is not sharp, and detecting it requires monitoring the ledger for coordinated patterns of contribution — a surveillance problem distinct from the attribution problem itself.

Policy attribution precision is limited by the difficulty of causal inference from observational data. When a data-sharing agreement was signed at the same time a model architecture was changed, the performance improvement that followed could be due to either or both. The ledger records timestamps and contribution identities, but it cannot enforce controlled experiments on the order of changes. Disentangling simultaneous contributions requires either accepting an approximation — attributing the joint effect proportionally to each contributor's estimated isolated effect — or designing the system to introduce changes sequentially rather than simultaneously. The latter is a governance recommendation, not a technical fix: the constitutional rules for when to introduce changes should be designed with attribution precision in mind.

Method attribution remains unresolved as a formal problem. The causal structure of a method document's influence is genuinely complex: it shapes not just one training run but the entire trajectory of the system's development, influencing what experiments are run, what data is collected, what features are engineered. Tracing this influence through the ledger's history is possible in principle — every training run records its active method — but quantifying its counterfactual effect requires specifying what would have happened in the method's absence, which is itself a research problem. We regard this as the most important open problem in the framework and anticipate that progress will come from combining ledger-based observational analysis with designed experiments in which method variations are deliberately introduced and their effects measured.

Cross-domain generalisation creates an attribution problem that the current framework cannot handle. A recommendation model trained on art data that is later deployed to recommend music or film generates value in domains not covered by the original data-sharing agreements. Art data contributors are not obviously entitled to compensation for value created in music — but they are also not obviously not entitled to it, if the art data enabled the initial model development that made the cross-domain transfer possible. A cross-domain Shapley extension would need to model the causal path from each domain's contributions to the value created in other domains, which requires both a richer ledger structure and a more complex value function. We flag this as an important direction without claiming to have a solution.

8. Conclusion

The Shapley value provides the correct theoretical foundation for fair attribution in systems where value is co-created by a heterogeneous set of contributors. Its four axioms — efficiency, symmetry, null player, and linearity — are not arbitrary desiderata but the minimal conditions for a distribution rule to be called fair in any meaningful sense. Data Shapley extends this foundation to machine learning, making it applicable to data contributions in recommendation systems and other ML-driven platforms.

The central contribution of this paper is to argue that the extension to code, policy decisions, and method documents follows naturally from the same framework, given one additional requirement: a provenance ledger that records the marginal effect of every contribution type on system performance. Without such a ledger, attribution is guesswork. With it, attribution becomes a structured query over a verifiable developmental history. The Filix ledger in HAAK is designed to provide exactly this infrastructure: a Merkle-CRDT record of every contribution, every training run, and every performance measurement, linked by cryptographic commitments that prevent retroactive falsification.

Ocean Protocol demonstrates that the pieces needed to operationalise this framework — data tokenisation, automatic distribution via smart contracts, privacy-preserving compute-to-data — are technically and legally feasible at scale. The gap between Ocean's current model and what we describe here is the treatment of governance decisions and method documents as first-class contributors alongside data and code. Closing that gap requires both the ledger infrastructure and the theoretical extension to heterogeneous contribution types that we have developed.

The immediate application is art intelligence platforms. Artists who deposit their work, galleries that contribute market data, auction houses that share transaction records, engineers who improve the recommendation algorithm, and researchers who develop the methods that make it work — all of these parties contribute to a shared system, and all of them have strong incentives to contribute more if they can trust that contribution will be compensated in proportion to its actual value. The framework described here is the mechanism that makes that trust possible.

References

Ghorbani, A., & Zou, J. (2019). Data Shapley: Equitable valuation of data for machine learning. *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 97, 2242–2251.

Koh, P. W., & Liang, P. (2017). Understanding black-box predictions via influence functions. *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 70, 1885–1894.

Kwon, Y., & Zou, J. (2022). Beta Shapley: A unified and noise-reduced data valuation framework for machine learning. *Proceedings of the 25th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 151, 8780–8802.

Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems (NeurIPS)*, 30, 4765–4774.

Shapley, L. S. (1953). A value for n-person games. In H. W. Kuhn & A. W. Tucker (Eds.), *Contributions to the Theory of Games, Volume II* (pp. 307–317). Princeton University Press.

Trentmann, N., Trautman, P., Köppel, M., & others (2022). Ocean Protocol: Tools for the Web3 Data Economy. *arXiv preprint arXiv:2203.16063*.

Generated by HAAK · Champalimaud Research · March 2026